

● 데이터 엔지니어 5기 라이브 스터디 · 4차시 과제

Flink/Paimon 적재를 증거로 설명하기

이번 과제의 목적은 "명령어를 실행했다"가 아니라, Kafka -> Flink -> Paimon 각 계층에서 어디까지 정상인지 증거로 설명하는 것입니다.

마감

2026년 6월 12일(금) 17:00

제출처

Discord 4차시 채널

통과 기준

필수 항목 제출. 실패 로그도 산출물로 인정

오늘 만든 흐름

Olist 기반 이벤트를 Kafka topic에서 읽고, Flink SQL streaming job으로 Paimon Bronze/current-state 테이블에 적재합니다.

1. Kafka

ux/review/order 이벤트를 topic에 보관

2. Flink SQL

Kafka source를 계속 구독하는 RUNNING job

3. Paimon

append table과 primary key table에 저장

4. MinIO

Paimon warehouse 파일을 object storage에 보관

5. Spark/Jupyter

snapshot, files, Iceberg-compatible metadata 관찰

핵심 구분: `ux_events_bronze` 는 행동 발생 사실을 계속 쌓는 append table이고, `review_current` 와 `order_current` 는 같은 key의 최신 상태를 유지하는 current-state table입니다.

먼저 실행할 명령어

아래 명령은 4차시 폴더에서 실행합니다.

```
cd de5-commerce-lakehouse-student/sessions/04-flink-paimon
git pull
cp .env.example .env # 이미 있으면 생략 가능
./scripts/check-env.sh
```

기준 실행 흐름

```
docker compose -f docker-compose.lite.yml up -d --build

./scripts/reset-olist-kafka-topics.sh
./scripts/reset-olist-paimon.sh
./scripts/run-flink-olist-paimon-streaming.sh

./scripts/produce-olist-ux-events.sh
./scripts/produce-olist-review-events.sh
./scripts/produce-olist-order-events.sh

./scripts/query-olist-paimon.sh
./scripts/query-olist-paimon-ops.sh
```

주의: `reset-olist-paimon.sh` 직후에 Jupyter나 query를 실행하면 table not found가 날 수 있습니다. reset은 테이블을 지우는 단계이고, `run-flink-olist-paimon-streaming.sh` 가 테이블을 다시 만들고 streaming job을 올리는 단계입니다.

필수 제출물

아래 항목까지 제출하면 통과입니다. 캡처만 올리지 말고, 각 캡처가 무엇을 증명하는지 한 줄씩 적어주세요.

1. 실행 결과 캡처

- `./scripts/check-env.sh` 결과
- `docker compose -f docker-compose.lite.yml ps` 결과
- `./scripts/run-flink-olist-paimon-streaming.sh` 실행 후 Job ID 3개 또는 Flink UI RUNNING 화면
- `./scripts/query-olist-paimon.sh` 결과
- 실패했다면 실행한 명령어와 에러 메시지

2. 기대 count 확인

아래 값과 본인 결과를 비교해주세요. 환경에 따라 streaming 처리 중에는 잠시 값이 덜 찰 수 있습니다. 조금 기다린 뒤 다시 조회해도 됩니다.

테이블	기대값	해석
<code>ux_events_bronze</code>	16,693	행동 이벤트 append
<code>review_current</code>	1,971	<code>review_id</code> 기준 current-state
<code>order_current</code>	2,000	<code>order_id</code> 기준 current-state

3. append vs upsert 설명

- `ux_events_bronze` 는 왜 append가 자연스러운가?
- `review_current` 는 왜 `review_id` primary key가 필요한가?
- `order_current` 는 왜 input event 수보다 row count가 작아지는가?

4. 검증 체인 표 작성

아래 표를 본인 결과로 채워주세요. 핵심은 "어디까지 정상인지"를 계층별로 분리하는 것입니다.

계층	내가 확인한 증거	정상 기준	이상할 때 다음 확인
Kafka	topic/message/offset	메시지 존재, offset 증가	producer log, topic, listener
Flink	RUNNING job, Records, checkpoint	job 3개 실행, source/sink 흐름 존재	Flink UI, task log, source/sink SQL
Paimon	row count, event 분포, raw_json	기대 count와 분포 일치	PK, warehouse path, raw payload

필수 질문

아래 질문에 본인 말로 짧게 답해주세요. 정답 문장을 외우는 과제가 아니라, 지금 구조를 스스로 설명하는 연습입니다.

1. **reset** 직후 **table not found**가 나는 이유를 설명해주세요.
2. **Flink job**이 **RUNNING**이라는 것과 **Paimon row count**가 **기대값**이라는 것은 각각 무엇을 증명하나요?
3. **raw_json**을 남기지 않았다면 JSON parsing이나 schema 변경 장애에서 무엇을 잃게 될까요?
4. **Paimon row count**가 **0**이라면 Kafka, Flink, Paimon 중 어디부터 어떤 순서로 확인할까요?

심화 선택

심화는 선택입니다. 한 가지 이상 해보면 5차시와 6차시에서 운영 관점으로 이어가기 좋습니다.

A. Snapshot / time travel 관찰

```
./scripts/query-olist-paimon-ops.sh
```

- `review_current$snapshots` 결과에서 `APPEND` 와 `COMPACT` 흐름을 확인
- 특정 snapshot id 기준 time travel 결과 확인
- "문제 직전 정상 기준점"으로 snapshot을 어떻게 쓸 수 있는지 3줄 정리

B. JupyterLab으로 Spark 관찰

```
docker compose -f docker-compose.lite.yml --profile query up -d --build jupyter
```

- `http://localhost:8888/lab` 접속
- `notebooks/de5-spark-starter.ipynb` 실행
- `Paimon $snapshots`, `VERSION AS OF`, `Iceberg snapshots` 중 하나 캡처

Jupyter가 table을 못 찾으면 먼저 streaming ingest가 실행됐는지 확인하고, 필요하면 커널을 재시작해 주세요.

C. MinIO에서 Paimon 파일 보기

- `http://localhost:9001` 접속
- 계정: `minioadmin` / 비밀번호: `minioadmin`
- `paimon` bucket 아래 `warehouse/bronze.db/` 경로 확인
- `snapshot -> manifest -> data file` 구조를 본인 말로 설명

D. 같은 데이터 replay 실험

produce 스크립트를 한 번 더 실행한 뒤 count 변화를 관찰합니다.

```
./scripts/produce-olist-ux-events.sh
./scripts/produce-olist-review-events.sh
./scripts/produce-olist-order-events.sh
./scripts/query-olist-paimon.sh
```

- `ux_events_bronze` 는 append라 증가할 수 있습니다.
- `review_current` , `order_current` 는 같은 key 기준으로 접혀 row count가 크게 변하지 않을 수 있습니다.
- 이 차이를 Kafka 중복 방지와 downstream upsert 관점에서 설명해보세요.

자주 만나는 오류

1. Could not find a file io implementation for scheme 's3'

Flink 이미지 안에 `paimon-s3` jar가 없는 상태입니다. 최신 코드 pull 후 Flink 이미지를 다시 빌드하세요.

```
docker compose -f docker-compose.lite.yml exec -T flink-jobmanager \
  bash -lc "ls -l /opt/flink/lib | grep paimon-s3"

docker compose -f docker-compose.lite.yml stop flink-jobmanager flink-taskmanager
docker compose -f docker-compose.lite.yml build --no-cache flink-jobmanager
docker compose -f docker-compose.lite.yml up -d --force-recreate flink-jobmanager f
```

2. Jupyter에서 TABLE_OR_VIEW_NOT_FOUND

reset 후 streaming ingest를 아직 실행하지 않았거나, Spark 커널이 이전 상태를 보고 있을 수 있습니다.

```
./scripts/run-flink-olist-paimon-streaming.sh
./scripts/query-olist-paimon.sh
```

그래도 안 되면 Jupyter 커널 재시작 후 다시 실행

3. JupyterLab 접속 불가

Jupyter는 기본 profile이 아니라 `query` profile로 띄웁니다. 8888 포트가 충돌하면 포트를 바꿔 실행하세요.

```
docker compose -f docker-compose.lite.yml --profile query up -d --build jupyter

# 8888 포트가 이미 사용 중이면
JUPYTER_HOST_PORT=18888 docker compose -f docker-compose.lite.yml --profile query up
```


제출 방식

Discord에는 긴 양식을 그대로 채울 필요가 없습니다. 본인이 편한 형식으로 이미지, PDF, Markdown, Notion 링크 등을 올려도 됩니다.

제출물은 아래 3가지만 확인되면 충분합니다. 중요한 것은 예쁘게 정리하는 것이 아니라, 실행 결과를 근거로 "어디까지 정상인지" 설명하는 것입니다.

1. 실행 증거

- Flink RUNNING job 3개
- `query-olist-paimon.sh` 결과
- 실패했다면 실행 명령어와 에러 메시지

2. 결과 해석

- 세 테이블 row count 비교
- append table과 current-state table의 차이
- reset 직후 table not found가 날 수 있는 이유

3. 검증 체인

- Kafka에서 본 증거
- Flink에서 본 증거
- Paimon에서 본 증거
- 문제가 생기면 어디부터 볼지

4. 심화 선택

- snapshot/time travel
- Jupyter/Spark 관찰
- MinIO 파일 구조
- 같은 데이터 replay 실험

이 과제의 핵심은 성공 화면 수집이 아니라, 각 계층의 증거를 연결해 "어디까지 정상인지" 설명하는 것입니다.